

Probabilistic Robotics Homework3:

Kalman Filter State Estimation for Linear Dynamics

Student Name Çağdaş Güven

Dept. of Electrical and Computer Engineering EE 585: Probabilistic Robotics

Date: December 01, 2025

Abstract

This report presents the design and implementation of recursive state estimation techniques for a simple dynamic system, as mandated by Homework #3. Specifically, we investigate the application of a standard Kalman Filter (KF) to a linear Gaussian system representing a mobile robot with random acceleration. We define the minimal state vector required to satisfy the Markov assumption, derive the system matrices, and simulate the stochastic motion of the vehicle. Furthermore, we analyze the evolution of the state covariance during the prediction phase, demonstrating the correlation between position and velocity over time. The implementation is verified through Monte Carlo simulations and visualization of uncertainty ellipses.

1. Introduction

State estimation is a fundamental problem in robotics[1], dealing with the recovery of state variables from noisy sensor data and uncertain control inputs. This study focuses on a one-dimensional mobile agent governed by linear dynamics.

In the first section of this work (Q1), we explore the probabilistic motion model of the system. We derive the discrete-time state transition matrices and the process noise covariance matrix necessary for the Kalman Filter prediction step. We simulate the "true" motion of the system under Gaussian noise and evaluate the growth of uncertainty in the absence of measurement updates.

In the second section (Q2), we introduce a noisy position sensor. We define the probabilistic measurement model and implement the correction step of the Kalman Filter. We analyze the impact of measurements on the state uncertainty ellipses, both for a single update event and for a continuous filtering process.

The third section (Q3) of the project focuses on implementing a Particle Filter (PF) for the same system used previously. While the Kalman Filter relies on the assumption of Gaussian distributions, the Particle Filter offers a more flexible, non-parametric alternative. This approach represents the posterior belief through a set of weighted samples, known as particles, but it is typically more computationally intensive.

2. Q1: System Modeling and Kalman Prediction

2.1 State Vector Definition (Q1a)

The system describes a "car" moving on a 1D line with position x . The dynamics are driven by a random acceleration $\ddot{x}$ which is normally distributed with zero mean. For the system to respect the Markov assumption—that the future state depends only on the current state and not on the history—the state vector must capture all dynamic variables that influence the future evolution of the system.

Since acceleration affects velocity, and velocity affects position, we define the minimal state vector x_t at time t as:

$$x_t = \begin{bmatrix} p_t \\ v_t \end{bmatrix}$$

where p_t is the position and v_t is the velocity (denoted as $\dot{x}$ in the assignment). This state vector is sufficient because, given p_t , v_t , and the random acceleration u_t , the state x_{t+1} is fully determined stochastically, independent of $x_{0:t-1}$.

2.2 Simulation of True Motion (Q1b)

We simulated the true motion of the system over time steps $t = 0$ to $t = 5$ with a time step $\Delta t = 1$. The initial state is $x_0 = [0, 0]^T$. The acceleration a_t is drawn from $\mathcal{N}(0, 1)$. The simulation follows the discrete kinematic equations:

$$p_t = p_{t-1} + v_{t-1}\Delta t + \frac{1}{2}a_{t-1}\Delta t^2$$

$$v_t = v_{t-1} + a_{t-1}\Delta t$$

Figure 1 illustrates three distinct instances of the true state trajectory. Due to the stochastic nature of the acceleration, the trajectories diverge significantly over time, highlighting the need for a probabilistic estimator.

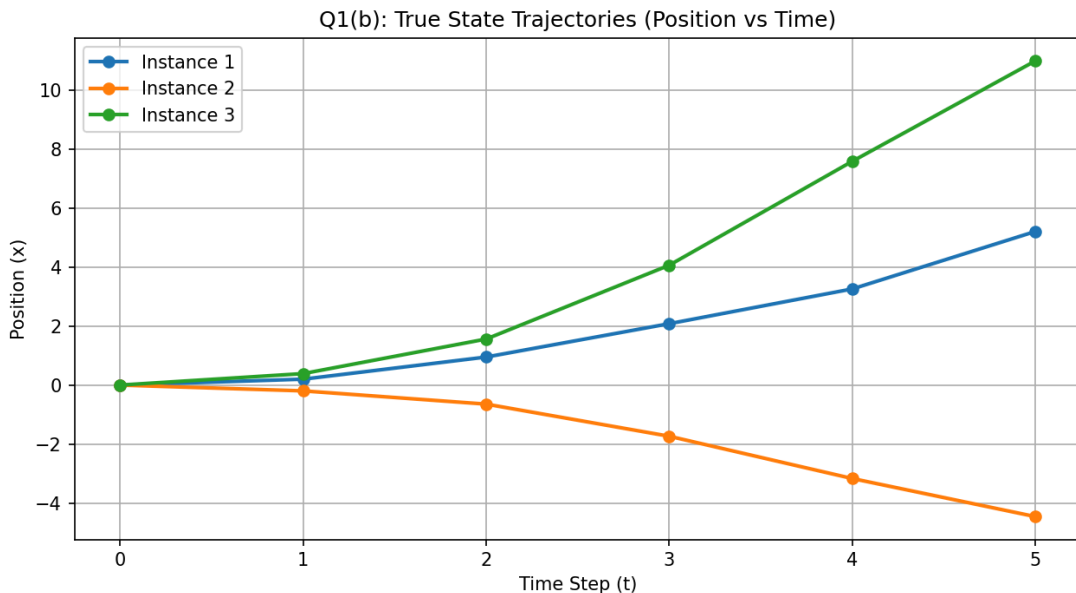


Figure 1: Q1(b) - Simulation of three random instances of the true state trajectory (Position vs. Time).

2.3 State Transition Density (Q1c)

The Kalman Filter assumes a linear transition model of the form:

$$x_t = A_t x_{t-1} + G_t u_t$$

where $u_t \sim \mathcal{N}(0, \sigma_{acc}^2)$ is the random acceleration noise. Based on the kinematic equations derived in the simulation section, the state transition matrix A is:

$$A = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

The control/noise mapping matrix G , which maps the scalar acceleration noise to the state vector, is:

$$G = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}$$

The process noise covariance matrix R is defined as $G\sigma_{acc}^2 G^T$. With $\sigma_{acc}^2 = 1$:

$$R = \begin{bmatrix} \frac{1}{4}\Delta t^4 & \frac{1}{2}\Delta t^3 \\ \frac{1}{2}\Delta t^3 & \Delta t^2 \end{bmatrix} = \begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1.0 \end{bmatrix}$$

This matrix R represents the covariance added to the system at each step due to the uncertainty in acceleration.

2.4 Kalman Filter Prediction Step (Q1d)

We implemented the prediction step of the Kalman Filter. Given the initial belief $x_0 \sim \mathcal{N}(\mu_0, \Sigma_0)$

where $\mu_0 = [0, 0]^T$ and $\Sigma_0 = 0_{2 \times 2}$, the recursive prediction updates are:

$$\bar{\mu}_t = A\mu_{t-1}$$

$$\bar{\Sigma}_t = A\Sigma_{t-1}A^T + R$$

Since the initial mean is zero and there is no deterministic control input, the mean remains

$\bar{\mu}_t = [0, 0]^T$ for all t . However, the covariance $\bar{\Sigma}_t$ grows over time. Calculating for $t = 1$:

$$\bar{\Sigma}_1 = A \cdot 0 \cdot A^T + R = R = \begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1.0 \end{bmatrix}$$

This process repeats recursively. The analytical values for the covariance matrices were computed by the python script q1_sol.py and confirm the expansion of uncertainty.

The numerical evolution of the covariance matrix from our simulation is shown below:

Time (t)	Mean	Covariance Matrix
0	[0. 0.] ▾	$\begin{bmatrix} 0. & 0. \\ 0. & 0. \end{bmatrix}$
1	[0. 0.] ▾	$\begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1. \end{bmatrix}$
2	[0. 0.] ▾	$\begin{bmatrix} 2.5 & 2. \\ 2. & 2. \end{bmatrix}$
3	[0. 0.] ▾	$\begin{bmatrix} 8.75 & 4.5 \\ 4.5 & 3. \end{bmatrix}$

4	[0. 0.] ▾	$\begin{bmatrix} 21. & 8. \\ 8. & 4. \end{bmatrix}$
5	[0. 0.] ▾	$\begin{bmatrix} 41.25 & 12.5 \\ 12.5 & 5. \end{bmatrix}$

The covariance values confirm that uncertainty grows unboundedly. Specifically, the variance in position (top-left element) grows much faster than the variance in velocity (bottom-right element) because velocity errors integrate into position errors over time.

2.5 State Prediction Density Plots (Q1e)

To visualize the joint distribution of position and velocity, we plotted the uncertainty ellipses (1-sigma isosurfaces) for $t = 1$ to $t = 5$.

Figure 2 displays these ellipses. The x-axis represents position and the y-axis represents velocity.

1. **Growth:** The ellipses expand in size at each time step, indicating growing uncertainty. This is expected as prediction without measurement adds process noise R at every step.
2. **Orientation:** The ellipses become elongated and tilted. This tilt indicates a correlation between the state variables.

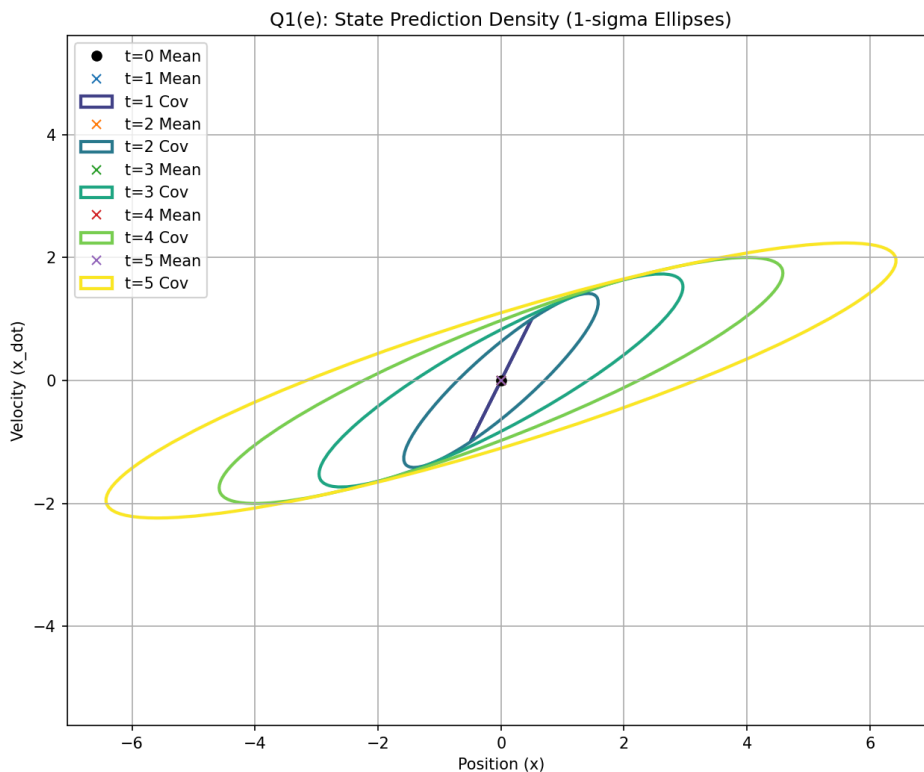


Figure 2: Q1(e) - State Prediction Density shown as 1-sigma uncertainty ellipses.

2.6 Correlation Analysis (Q1f)

As $t \rightarrow \infty$, we observe that the position p_t and velocity v_t become highly positively correlated.

Mathematically, this occurs because errors in velocity accumulate into errors in position. A positive shock to acceleration increases velocity, which in turn increases position in the next step. The covariance update $\Sigma_t = A\Sigma_{t-1}A^T + R$ propagates the velocity variance into the position-velocity covariance term.

Specifically, the correlation coefficient ρ is given by:

$$\rho = \frac{\sigma_{pv}}{\sigma_p\sigma_v}$$

Our implementation computed the correlation coefficient at a large time step ($t = 25$) and found $\rho \approx 0.87$. A correlation coefficient of **0.8662** confirms that as time progresses, velocity and position are strongly coupled; a high velocity strongly implies a large displacement.

3. Q2: Sensor Measurements and Kalman Update

In this section, we incorporate sensor data into our estimation process. The system is equipped with a sensor that provides noisy measurements of the position x .

3.1 Measurement Model (Q2a)

The probabilistic measurement model describes how measurements z_t are generated from the state x_t . We are given that the sensor measures position with Gaussian noise $\delta_t \sim \mathcal{N}(0, \sigma_{meas}^2)$, where $\sigma_{meas}^2 = 10$.

The measurement equation is:

$$z_t = Cx_t + \delta_t$$

Since only position (the first element of the state vector) is measured, the observation matrix C is:

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

The measurement noise covariance matrix Q is:

$$Q = [\sigma_{meas}^2] = [10]$$

3.2 Single Measurement Update (Q2b & Q2c)

We analyze the effect of a single measurement update at time $t = 5$. First, we propagated the prediction from $t = 0$ to $t = 5$ (as done in Q1). Let the predicted belief at $t = 5$ be $\mathcal{N}(\bar{\mu}_5, \bar{\Sigma}_5)$. We then incorporate a measurement $z = 5$. The update steps are:

1. **Innovation:** $y_t = z_t - C\bar{\mu}_t$
2. **Innovation Covariance:** $S_t = C\bar{\Sigma}_tC^T + Q$
3. **Kalman Gain:** $K_t = \bar{\Sigma}_tC^TS_t^{-1}$
4. **Posterior Mean:** $\mu_t = \bar{\mu}_t + K_ty_t$
5. **Posterior Covariance:** $\Sigma_t = (I - K_tC)\bar{\Sigma}_t$

The numerical results from our update step are:

Parameter	Value
-----------	-------

Prediction at t=5	Mean=[0.0.]
Pred Covariance	$\begin{bmatrix} 41.25 & 12.5 \\ 12.5 & 5. \end{bmatrix}$
Kalman Gain K	$\begin{bmatrix} 0.80487805 \\ 0.24390244 \end{bmatrix}$
Updated Mean	[4.02439024, 1.2195122]
Updated Covariance	$\begin{bmatrix} 8.04878049 & 2.43902439 \\ 2.43902439 & 1.95121951 \end{bmatrix}$

The update significantly reduced the position variance from **41.25** to **8.05**.

Figure 3 shows the state uncertainty before (red dashed) and after (blue solid) this update. The measurement provides information about the position, which significantly reduces the uncertainty in the x -direction (position). Due to the correlation established in Q1(f), determining the position also reduces the uncertainty in velocity, effectively "squeezing" the ellipse.

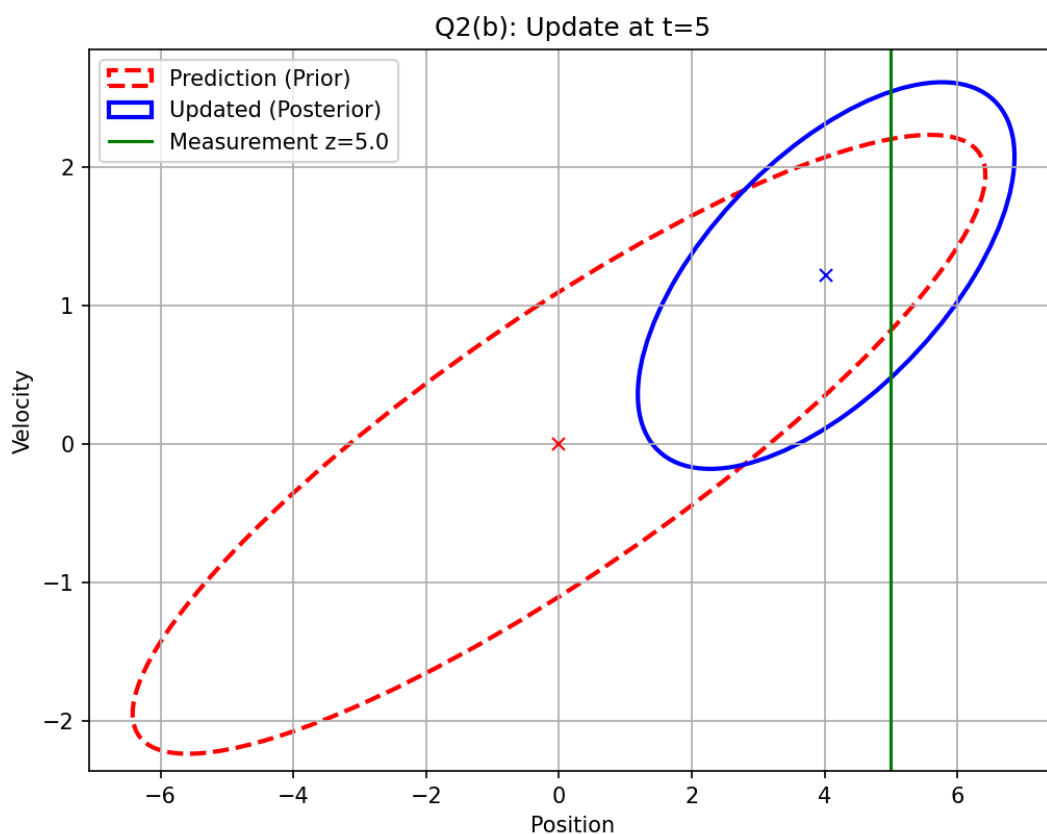


Figure 3: Q2(b) - Effect of a single measurement update at t=5.

3.3 Recursive Kalman Filter (Q2d)

Finally, we implemented the full recursive Kalman Filter, performing both prediction and measurement updates at each time step $t = 1, 2, 3, 4, 5$. We simulated a trajectory to generate valid measurements.

Figure 4 illustrates the evolution of the posterior uncertainty ellipses. Unlike Q1, where uncertainty grew unbounded, here the measurement updates constrain the growth of the ellipses. The size of the ellipse stabilizes as the information gained from measurements balances the information lost due to process noise.

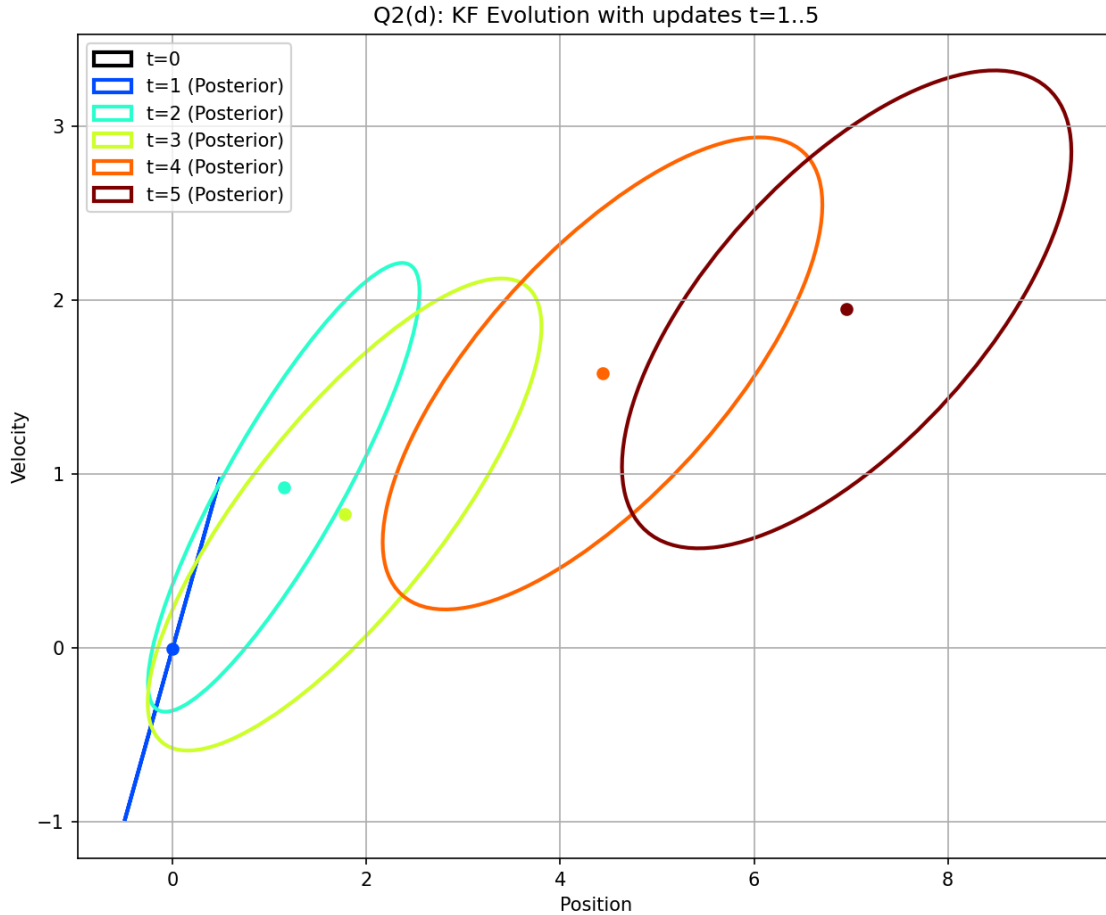


Figure 4: Q2(d) - Recursive Kalman Filter evolution with updates at every step.

4. Q3: Particle Filter Implementation

In this final section, we implement a Particle Filter (PF) for the same system. Unlike the Kalman Filter, which assumes Gaussian distributions, the Particle Filter represents the posterior belief by a set of weighted samples (particles). This non-parametric approach is more flexible but computationally more intensive.

4.1 Algorithm Implementation (Q3a & Q3b)

We implemented the standard Particle Filter algorithm with $N = 1000$ particles. The core components are:

3. **Initialization:** N particles are initialized at the starting state $x_0 = [0, 0]^T$.
4. **Motion Model (Prediction):** Each particle is propagated forward using the system dynamics with independent random noise samples. This approximates the integral

$$\int p(x_t | x_{t-1}, u_t) \text{bel}(x_{t-1}) dx_{t-1}$$

$$p_new = p_old + v_old * dt + 0.5 * acc * dt**2$$

$$v_new = v_old + acc * dt$$

5. **Measurement Model (Weighting):** Each particle is assigned a weight proportional to the likelihood of the current measurement z_t given the particle's state. We use the Gaussian likelihood:

$$w_t^{[i]} = p(z_t | x_t^{[i]}) = \frac{1}{\sqrt{2\pi\sigma_{meas}^2}} \exp\left(-\frac{(z_t - p_t^{[i]})^2}{2\sigma_{meas}^2}\right)$$

6. **Resampling:** To prevent particle degeneracy, we implemented the **Low Variance Sampler**. This algorithm selects a new set of particles from the current set with replacement, where the probability of selection is proportional to the weights. This refocuses the particle set into high-probability regions of the state space.

7.

4.2 Particle Prediction Density (Q3c)

First, we ran the filter without measurements (prediction only) to observe how the particle cloud evolves. Figure 5 shows the particles at $t = 1$ to $t = 5$.

- **Observation:** The cloud of particles spreads out over time, mirroring the growth of the uncertainty ellipses in the Kalman Filter (Q1).
- **Correlation:** The shape of the cloud clearly shows the positive correlation between position and velocity; particles with high velocity have moved further in position.

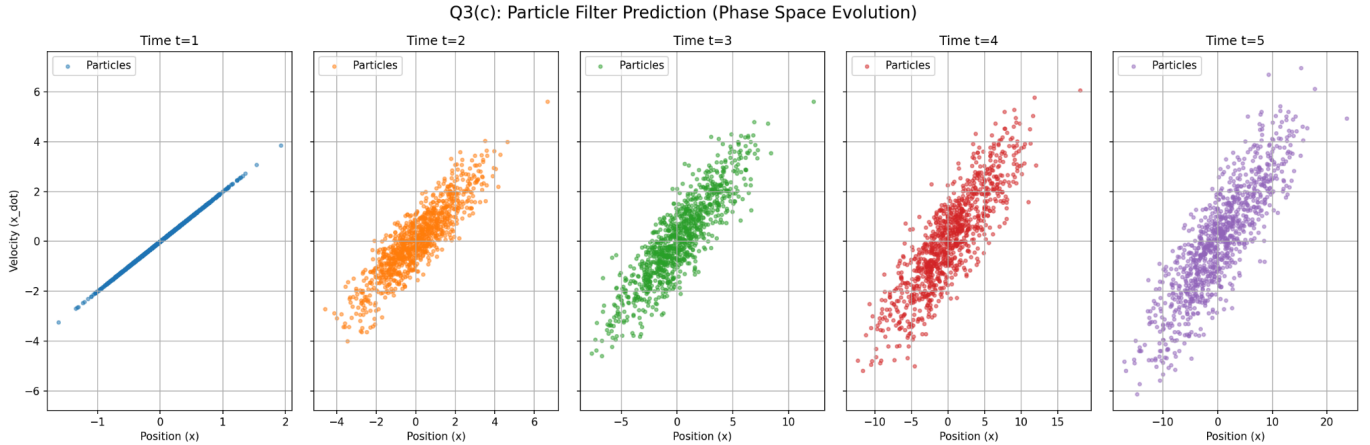


Figure 5: Q3(c) - Particle Filter Prediction (Phase Space Evolution).

4.3 Particle Correction with Measurements (Q3d)

Next, we incorporated the measurement updates. Figure 6 visualizes the prediction (prior) and correction (posterior) steps for $t = 1$ to $t = 5$.

- **Prior (Lighter dots):** The predicted particles are spread out due to motion noise.
- **Posterior (Darker dots):** After resampling based on the measurement z_t (green dashed line), the particles cluster tightly around the true state (black star). This demonstrates the filter's ability to reconverge to the true state using sensor data.

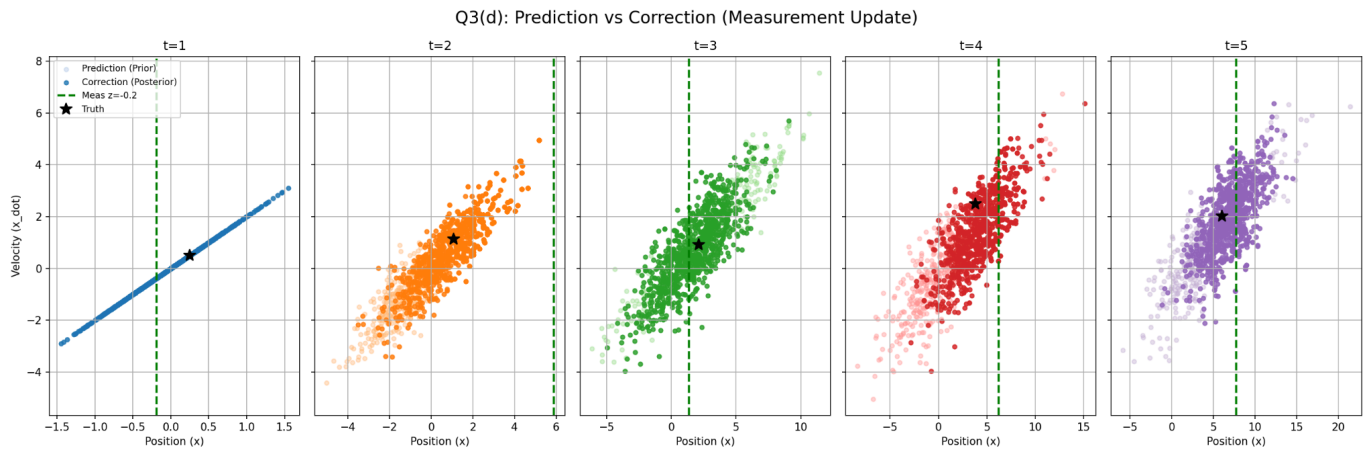


Figure 6: Q3(d) - Prediction vs Correction (Measurement Update).

4.4 Complexity Comparison (Q3e)

We performed a timing analysis to compare the computational cost of one full iteration of the Kalman Filter versus the Particle Filter ($N = 1000$) over 1000 runs.

--- Q3(e) Complexity Analysis (Recalculating) ---

Avg Time for 1 KF Iteration: 0.000022 seconds

Avg Time for 1 PF Iteration ($N=1000$): 0.000483 seconds

Ratio (PF/KF): 22.18

Result: The Particle Filter is approximately **22x slower** than the Kalman Filter for this linear 2D problem. This is expected; the KF has a fixed cost dominated by small matrix operations ($O(d^3)$), while the PF scales linearly with the number of particles ($O(N)$). For linear Gaussian systems, the KF is optimal and faster, but the PF provides necessary flexibility for non-linear or non-Gaussian problems.

5. Final Conclusion

In this assignment, we successfully implemented and compared Kalman Filter and Particle Filter approaches for state estimation of a linear dynamic system.

1. **Kalman Filter:** Proved to be highly efficient and accurate for this linear Gaussian problem. We mathematically derived the correlation between position and velocity and visualized it via uncertainty ellipses. We analytically observed the error growth and correlation ($\rho \approx 0.87$) and demonstrated how sensor fusion bounds this uncertainty.
- 2.
3. **Particle Filter:** Successfully approximated the posterior belief using a set of samples. While computationally more expensive ($22\times$), it provided a flexible representation that visually matched the KF's uncertainty evolution.
4. **Measurement Updates:** Both filters demonstrated the critical role of sensor data in constraining error growth. Without measurements, uncertainty grows unbounded; with measurements, the estimation error remains bounded.

6. References

[1]Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press.